

0.11.2. Tout l'alphabet, avec textS — doublé, deux positions

Tout l'alphabet deux fois : lignes 0 et 1, puis lignes 3 et 4.

1. Ce qu'il faut comprendre

C'est le 0.11 et le 0.11.1 réunis : l'alphabet du 0.11 (lignes 0 et 1), puis celui du 0.11.1 (lignes 3 et 4).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <textS>` → [0.10.3. La fonction textS\(\) — un texte qui passe à la ligne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **687 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         51 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         416 o ← CopierTuiles – 26 lettres sur 43
"ABCDEFGHIIJKLMNO..." 20 o ← texte() – un texte gravé
"UVWXYZ"       6 o ← texte() – un texte gravé
"ABCDEFGHIIJKLMNO..." 20 o ← texte() – un texte gravé
"UVWXYZ"       6 o ← texte() – un texte gravé
```

Lettres gravées : ABCDEFGHIIJKLMNOPQRSTUVWXYZ

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 4    44 o sur place, dans main() + EcrireTexte (10 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
textS() × 2    0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

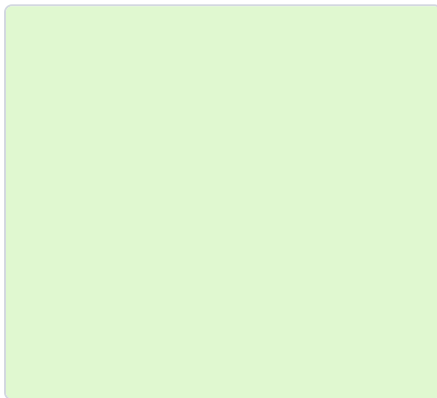
```
#include <textS> // écrit un texte qui passe à la ligne tout seul
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

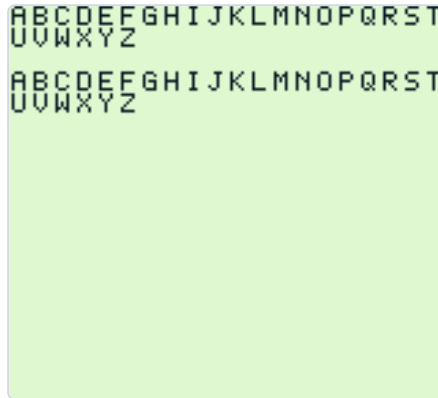
```
int main() {  
    // 1. Lignes 0 et 1 (le 0.11)  
    textS(0, 0, "ABCDEFGHJKLMNOPQRSTUVWXYZ"); // passe à la ligne 1 après T  
    // 2. Lignes 3 et 4 (le 0.11.1)  
    textS(0, 3, "ABCDEFGHJKLMNOPQRSTUVWXYZ"); // passe à la ligne 4 après T  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Tout l'alphabet deux fois : en haut, et trois lignes plus bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ lignes 0 et 1

✓ lignes 3 et 4

Ensuite : **0.12. Une lettre toutes les secondes** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++